

Project 1 - Data Warehouse Design

Project 1 submitted by

Butakov Kirill, Student ID: 24620697.

Dmitry Prytkov, Student ID: 24069389.

The overall objectives of this project are to build a data warehouse using real-world datasets and to carry out a basic data mining activity, in this case, association rule mining.

Datasets and Problem Domain

The data warehouse is constructed to enable multi-dimensional analysis of Australian road traffic fatalities in order to enable stakeholders to make data-driven, informed decisions to reduce road deaths and enhance traffic safety.

Data Sources

Two datasets from the Australian Road Deaths Database (ARDD) were initially considered for this project:

1. ARDD: Fatal crashes—December 2024—XLSX
2. ARDD: Fatalities—December 2024—XLSX

Fatalities dataset was found to be more comprehensive, as it not only includes information about the crashes but also includes detailed information about the victims. But the Fatal Crashes dataset lacks victim-specific information. Therefore, we have decided to make use of only the Fatalities dataset while building the data warehouse. The dataset is publicly available at:

https://www.bitre.gov.au/sites/default/files/documents/bitre_fatalities_dec2024.xlsx

We also employ available population data from the Australian Bureau of Statistics (ABS). ABS presents demographic figures in terms of various types of geographic areas and across various years. This supports analysis of death rates based on population size. The population dataset is available here:

[Population estimates by LGA, SUA, Remoteness Area, etc. \(2001–2023\)](#)

Data Warehousing Design and Implementation

Modelling process identification

The unit or process of data to be processed in this project is death of an individual who died as a result of a road traffic accident. The dataset has the following attributes that describe each fatality:

- **Crash ID:** National crash identification number
- **State:** Australian jurisdiction
- **Month:** Month of the crash
- **Year:** Year of the crash
- **Dayweek:** Day of the week the crash occurred
- **Time:** Time of the crash

- **Crash Type:** Refers to the number of vehicles involved in the crash
- **Bus Involvement:** Indicates whether a bus was involved in the crash
- **Heavy Rigid Truck Involvement:** Indicates whether a heavy rigid truck was involved in the crash
- **Articulated Truck Involvement:** Indicates whether an articulated truck was involved in the crash
- **Speed Limit:** Posted speed limit at the crash location
- **Road User:** Type of road user who was killed
- **Gender:** Sex of the deceased individual
- **Age:** Age of the deceased individual (in years)
- **National Remoteness Areas:** ABS Remoteness Structure
- **SA4 Name 2021:** Australian Statistical Geography Standard (ASGS) level
- **National LGA Name 2021:** Australian Statistical Geography Standard (ASGS) level
- **National Road Type:** Classification of the road type, based on Geoscape Australia, Transport, and Topography
- **Christmas Period:** Indicates if the crash occurred during the 12-day period starting on December 23rd
- **Easter Period:** Indicates if the crash occurred during the 5-day period starting on the Thursday before Good Friday
- **Age Group:** Standard age groupings used in the Road Deaths Australia monthly bulletin
- **Day of Week:** Indicates if the crash occurred on a weekday or weekend. (Note: 'Weekday' refers to Monday 6am through Friday 5:59pm)
- **Time of Day:** Indicates if the crash occurred during the day or night. (Note: 'Day' refers to 6am through 5:59pm)

Upon reviewing the data, we can identify several business questions that our data warehouse could help answer. For example:

1. What is the total number of road fatalities by state and year?
2. How many road crashes involving heavy vehicles occurred during the day and night across each state?
3. How many road crashes occurred during holidays in each state in 2024?
4. Which age group is most frequently involved in road crashes during rush hours?
5. How many road crashes occurred at each speed limit in each Australian state during 2024?
6. What is the road fatality rate per 100,000 people by state in the year 2020?

The capabilities of the designed data warehouse are not limited to these queries alone.

Determine the dimension tables.

In our case, we will use a Galaxy schema with two fact tables. The second fact table is necessary to store population data broken down by year, allowing us to analyze road mortality trends over time.

We define the following dimensions in our data warehouse schema, along with their respective hierarchies:

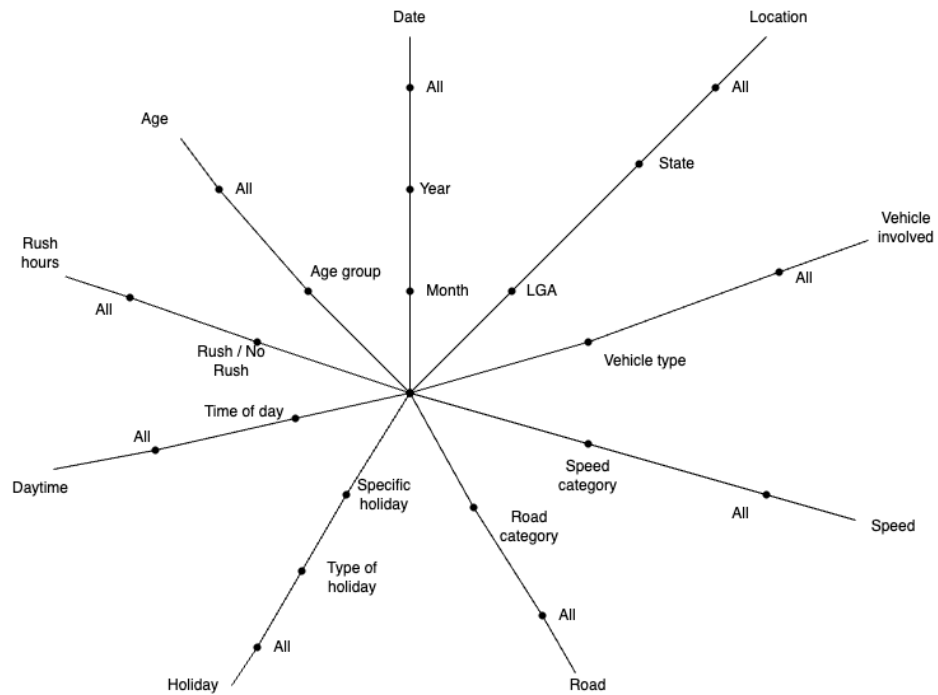
1. **Date:** Information about the date of the event, consisting of the year and month.

2. **Location:** Information about the location of the event, consisting of the state and Local Government Area (LGA).
3. **Vehicle Involved:** Information about the type of transport involved in the crash. This is a categorical variable with two values:
 - **Heavy Vehicle Involved:** If the crash involved a bus, heavy rigid truck, or articulated truck.
 - **No Heavy Vehicle Involved:** If no heavy vehicle was involved in the crash.
4. **Speed:** Information about the speed limit at the location of the crash.
5. **Road:** Information about the category of the road where the crash occurred.
6. **Holiday:** Information about whether the crash occurred during a holiday period. This includes the holiday type (Holiday, NoHoliday) and holiday name (Christmas, Easter or NoHoliday indicating regular days).
7. **Daytime:** Information about whether the crash occurred during the day or night.
8. **Rush Hours:** Information about whether the crash occurred during rush hours.
9. **Age:** Information about the age of the victim.

We do not use the following categories for analysis: **'age'**, **'road_user'**, **'gender'**, **'national_remoteness_areas'**, **'sa4_name_2021'**, and **'day_of_week'** for several reasons:

1. **Age:** Instead of using the exact numeric age values, we categorize individuals into **age groups**. This approach is preferred because using precise age values would significantly increase the size of the dimension table, potentially hindering the efficiency of analysis and insight generation.
2. **Road User:** The **road user** dimension is not included in the data warehouse because it does not align with the primary business queries. However, it is used in association rule mining, which helps identify patterns in specific types of road users involved in accidents.
3. **Gender:** The **gender** dimension is not included in the data warehouse because it does not align with the primary business queries.
4. **SA4 Name 2021, National Remoteness Areas:** The analysis does not focus on geographical remoteness as a primary factor or on the SA4 statistical areas. Instead, we focus on Local Government Areas (**LGA**), which is areas within states.
5. **Day of Week:** Rather than focusing on the **day of the week**, we choose categories such as **holidays**, **weekdays/weekends**, and **rush hours**. These categories are considered to have a more significant impact on the likelihood of accidents and offer more useful insights for road safety analysis.

Star scheme including schema hierarchies:

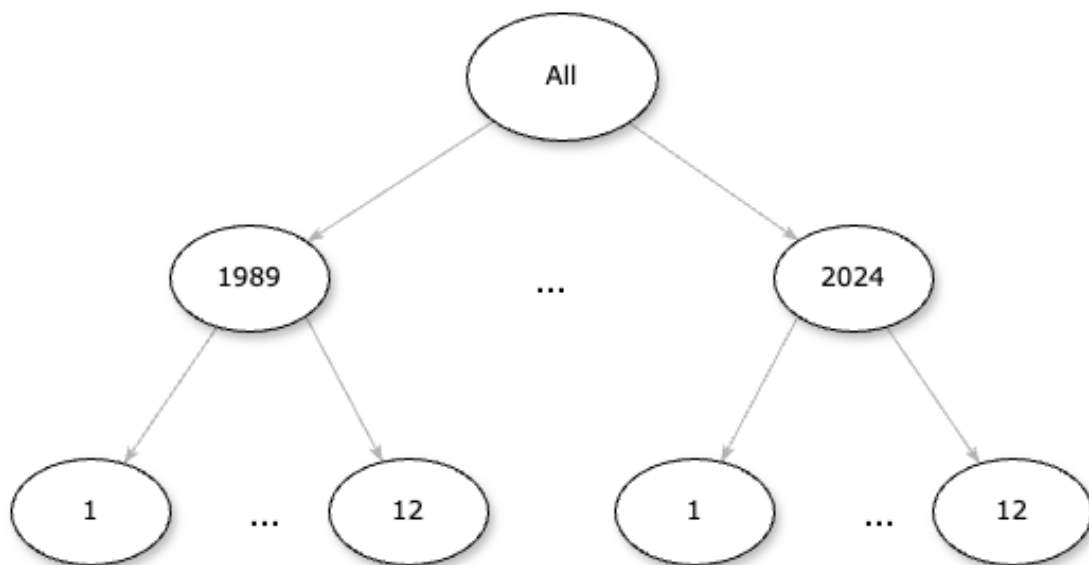


Concept hierarchies for dimensions shown below:

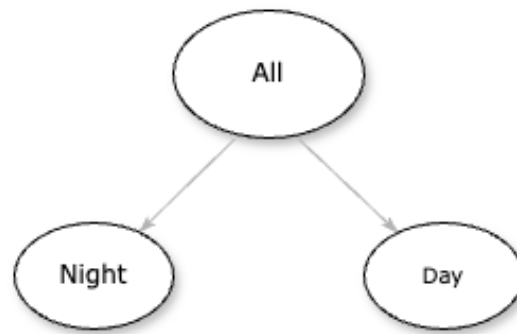
Concept Hierarchy - Age



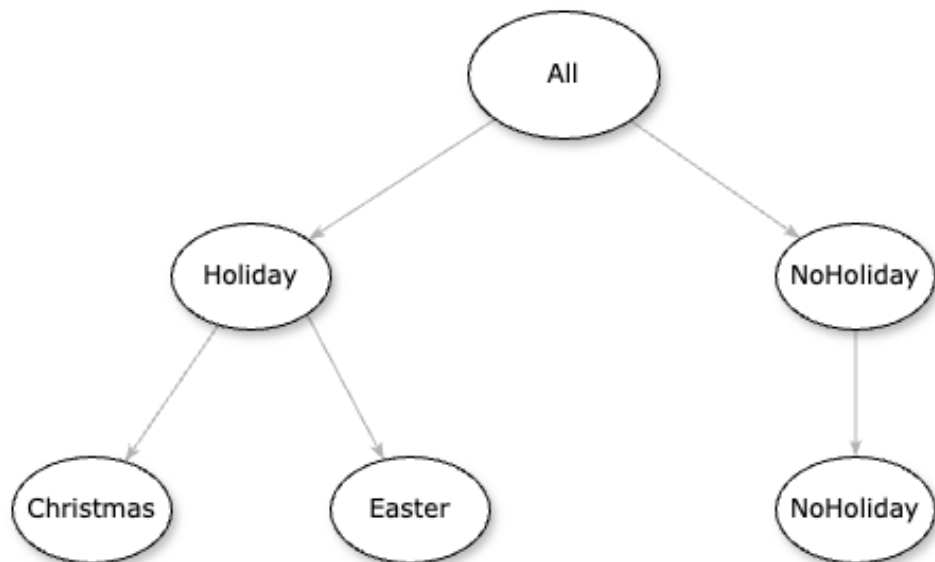
Concept Hierarchy - Date



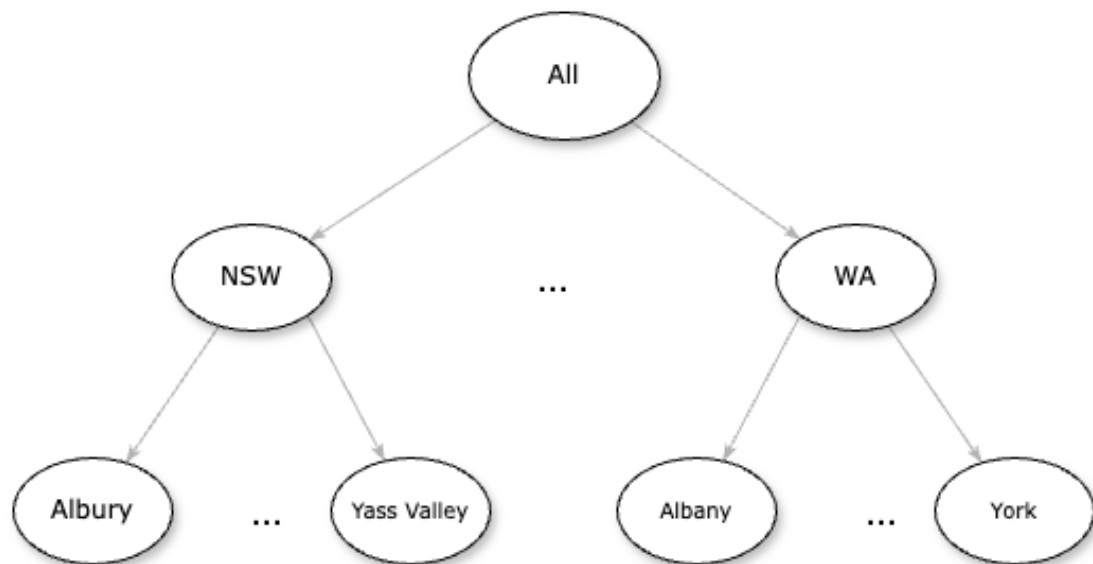
Concept Hierarchy - Daytime



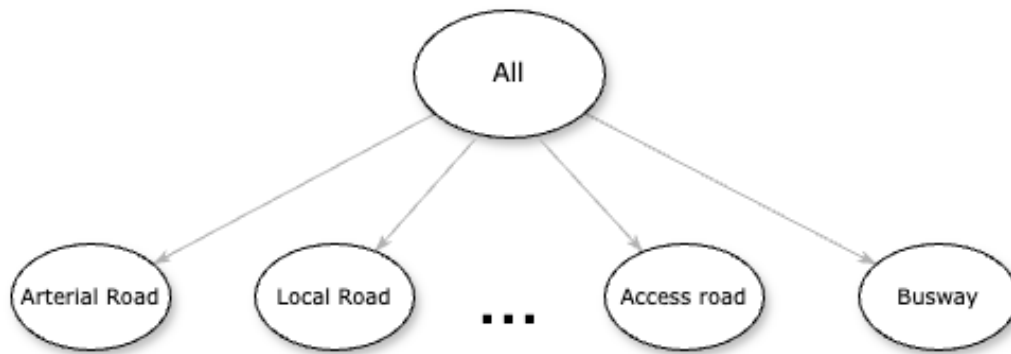
Concept Hierarchy - Holiday



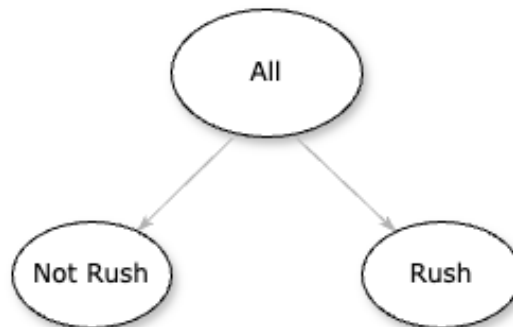
Concept Hierarchy - Location



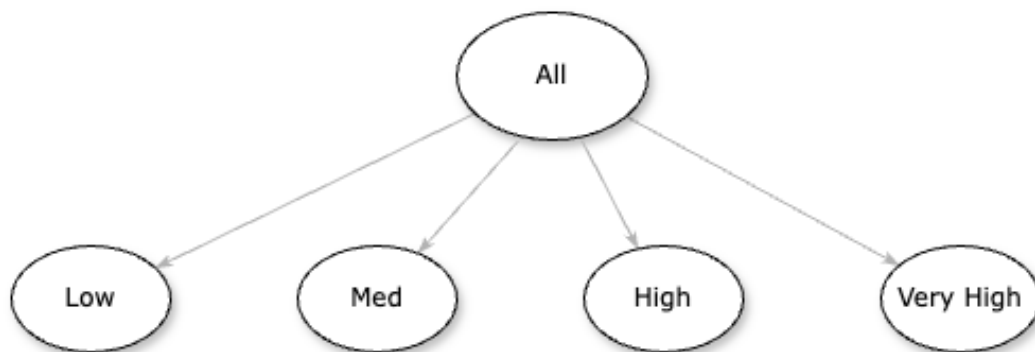
Concept Hierarchy - Road



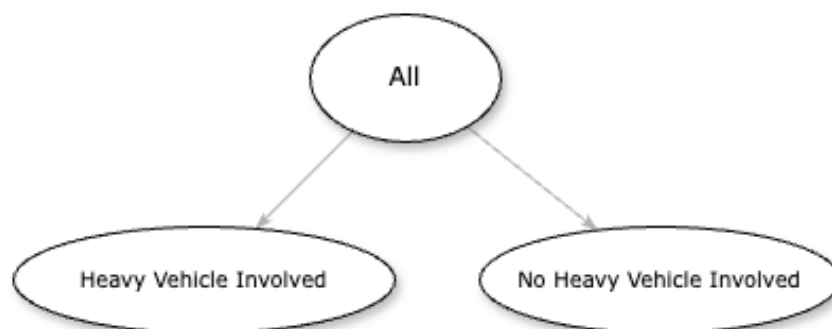
Concept Hierarchy - Rush hours



Concept Hierarchy - Speed

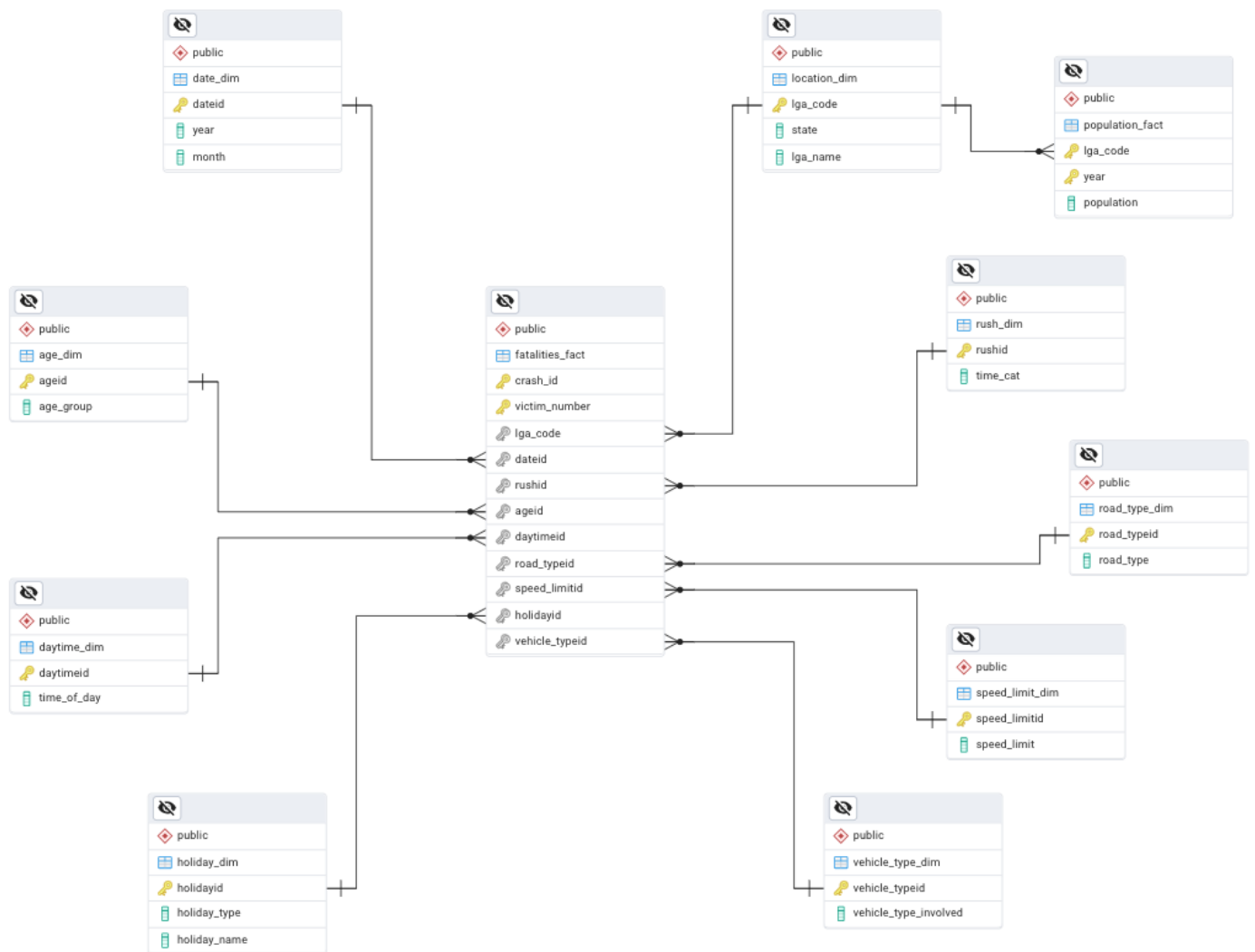


Concept Hierarchy - Vehicle involved



Schema, Starnet and query footprints

Data warehouse schema shows schema of fact and dimension tables along with relationships between them. For example, both fact tables have one common dimension table - location dimension table.



Determine the grain at which facts can be stored.

The first fact table is **fatalities_fact** table. It has no numeric measures so it is factless fact table, which will describe fatalities itself. Attributes for this fact table will be:

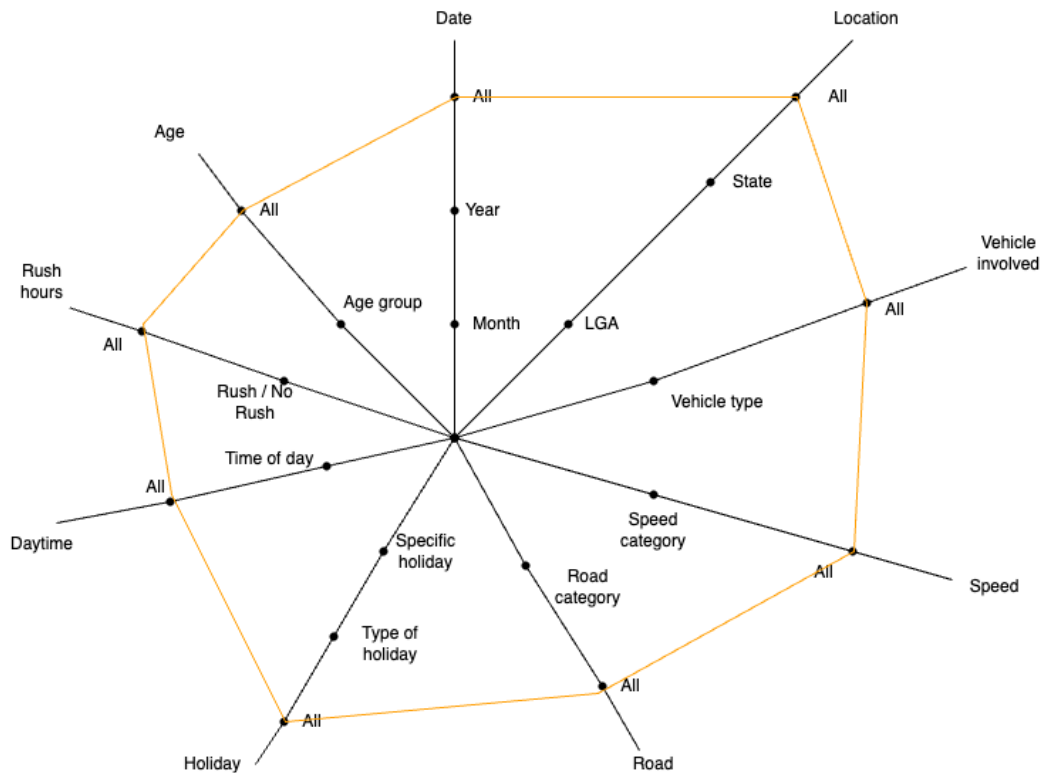
- **crash_id** - National crash identifying number.
- **victim_number** - Victim number in the accident. Together with crash_id forms a composite Primary key.
- **lga_code** - foreign key for location dimension.
- **dateID** - foreign key for date dimension.
- **rushID** - foreign key for Rush hours dimension.
- **ageID** - foreign key for Age dimension.
- **daytimeID** - foreign key for Daytime dimension.
- **road_typeID** - foreign key for Road dimension.
- **speed_limitID** - foreign key for Speed dimension.
- **holidayID** - foreign key for Holiday dimension.
- **vehicle_typeID** - foreign key for Vehicle involved dimension.

The second fact table is **population_fact** table with population measure. Attributes for this fact table will be:

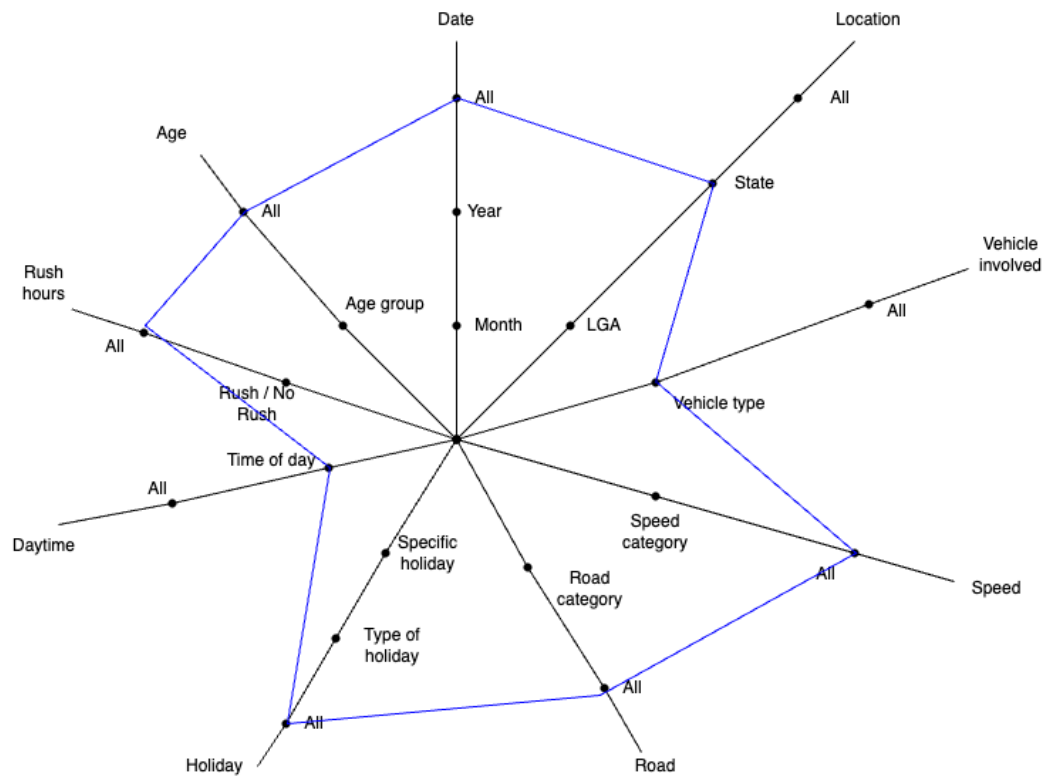
- **lga_code** - foreign key for location dimension.
- **year** - year when event occurred. Together with lga_code forms a composite Primary key.
- **population** - population measure.

Below are StarNet footprints to illustrate how the business queries can be answered with our Data Warehouse design.

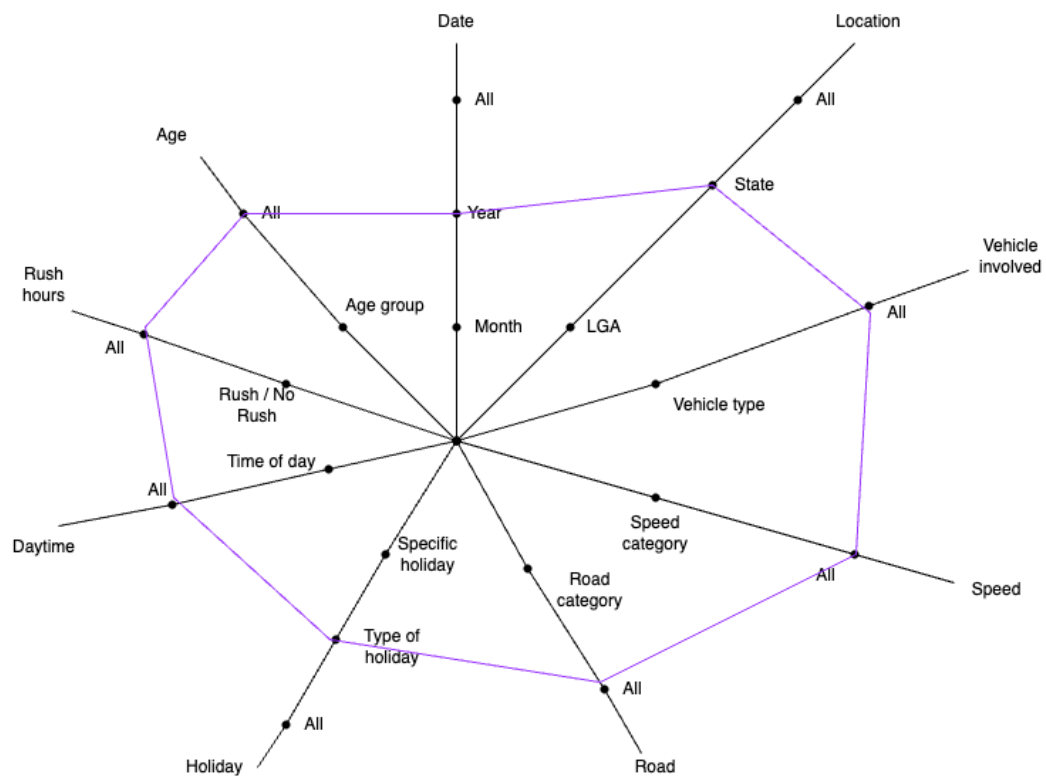
1. What is the total number of road fatalities by state, and year?



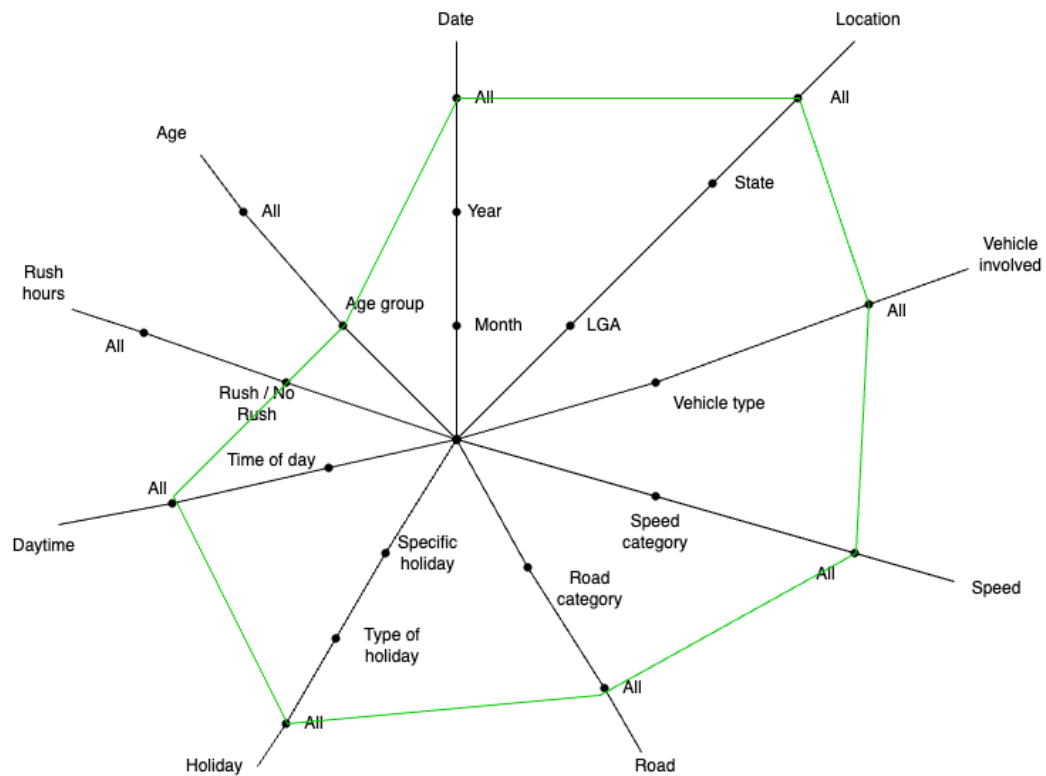
2. How many road crashes involving heavy vehicles occurred during the day and night across each state?



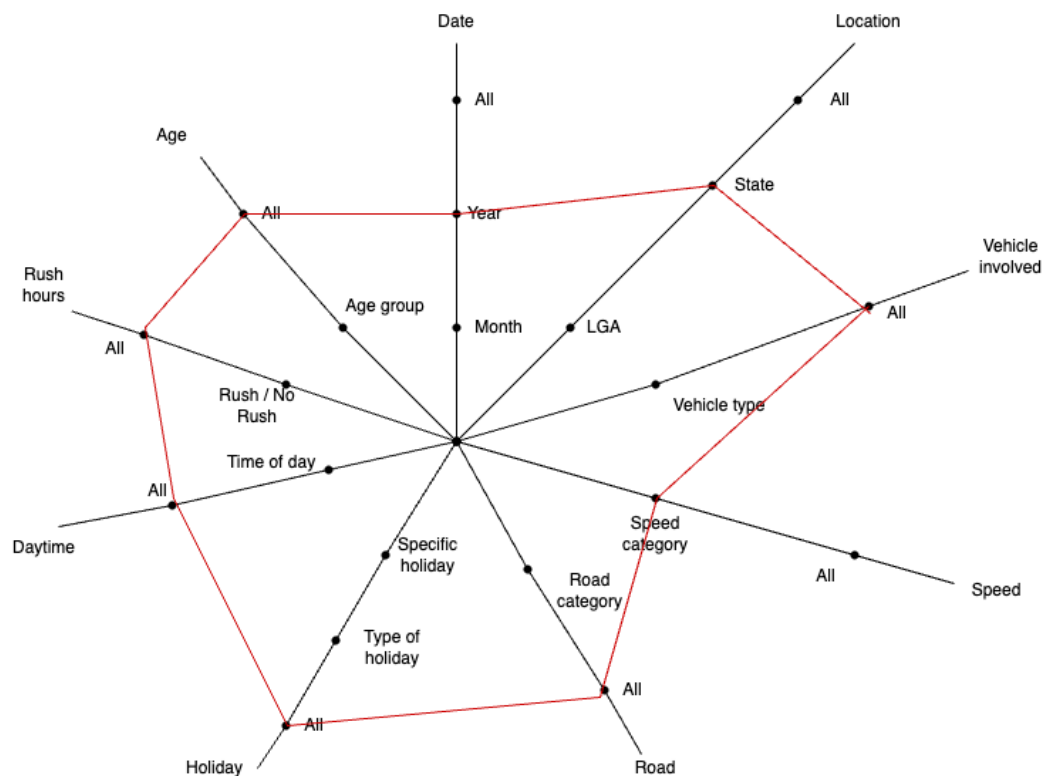
3. How many road crashes occurred during holidays in each state in 2024?



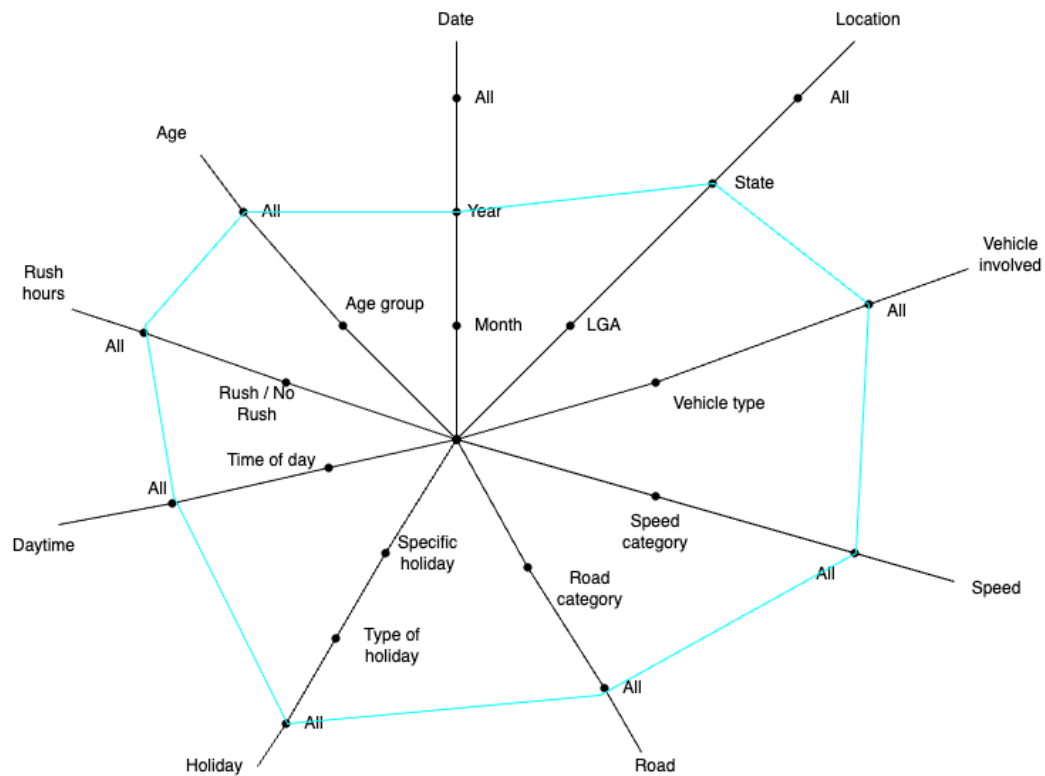
4. Which age group is most frequently involved in road crashes during rush hours?



5. How many road crashes occurred at each speed limit in each Australian state during the year 2024?



6. What is the road fatality rate per 100,000 people by state in the year 2020?



ETL

Extraction

Importing libraries.

```
import pandas as pd
import numpy as np
```

Reading Files.

```
# # Ensure the required library is installed
# (Can we do it (openpyxl)? Will it be a problem?)
# %pip install openpyxl

# Read the Excel files
file_path = "bitre_fatalities_dec2024.xlsx"
df = pd.read_excel(file_path, sheet_name="BITRE_Fatality", skiprows=4)

file_path = "Population.xlsx"
population = pd.read_excel(file_path, sheet_name="Table 1", skiprows=5)
```

Transformation

Changing column names for convinience.

```
# Clean the columnnames
```

```
# Remove leading and trailing whitespace, convert to lowercase, and replace spaces with
underscores

df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')

# change sa4_name_2021 to sa4_name, national_lga_name_2021 to lga_name
df.rename(columns={
    'national_lga_name_2021': 'lga_name',
    'national_road_type': 'road_type'
}, inplace=True)
df.columns

Index(['crash_id', 'state', 'month', 'year', 'dayweek', 'time', 'crash_type',
      'bus_involvement', 'heavy_rigid_truck_involvement',
      'articulated_truck_involvement', 'speed_limit', 'road_user', 'gender',
      'age', 'national_remoteness_areas', 'sa4_name_2021', 'lga_name',
      'road_type', 'christmas_period', 'easter_period', 'age_group',
      'day_of_week', 'time_of_day'],
      dtype='object')
```

Adding a victim number for each crash to make composite primary key.

```
df['victim_number'] = df.groupby('crash_id').cumcount() + 1

# move the victim_number column to the front
cols = df.columns.tolist()
cols.insert(1, cols.pop(cols.index('victim_number')))
df = df[cols]
```

Drop unnecessary columns.

```
df.drop(columns=['age', 'gender', 'national_remoteness_areas', 'sa4_name_2021',
                'day_of_week'], inplace=True)
```

Categorising variables

Create variable time_cat for **Rush hours** dimension. Rush hours is morning hours from 07:00 - 09:00 and evening hours 16:00 - 18:00 during working days.

```
# convert the 'time' column to datetime format
df['time'] = pd.to_datetime(df['time'], format='%H:%M:%S', errors='coerce').dt.time

Weekday = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]

conditions = [
    (df['christmas_period'] == "No") & (df['easter_period'] == "No") &
    (df['dayweek'].isin(Weekday)) & (
        ((df['time'] >= pd.to_datetime("07:00:00", format='%H:%M:%S').time()) & (df['time'] <=
pd.to_datetime("09:00:00", format='%H:%M:%S').time())) |
```

```

    ((df['time'] >= pd.to_datetime("16:00:00", format='%H:%M:%S').time()) & (df['time'] <=
pd.to_datetime("18:00:00", format='%H:%M:%S').time()))
    ),
    (df['time'].isna())
]

choices = ["Rush", np.nan]

df['time_cat'] = np.select(conditions, choices, default="Not Rush")

# change 'nan' value to np.nan
df['time_cat'] = df['time_cat'].replace('nan', np.nan)

print(df['time_cat'].unique())

['Not Rush' 'Rush' nan]

```

Set NaN for all **missing values** in dataset. We decided to leave all Nan values since there are no meaningful option to replace in with mean or mode value, because every crash is unique event.

```

nan_values = ['Other/-9', '-9', 'Unknown', 'Undetermined', -9]
df.replace(nan_values, np.nan, inplace=True)

```

Checking for 'bad values' in speed_limit variable.

```

try :
    df['speed_limit'] = df['speed_limit'].astype(float)
except ValueError:
    # If conversion to int fails, print values that cannot be converted
    print("Values that cannot be converted to int:")
    print(df[~df['speed_limit'].apply(lambda x: isinstance(x, int) or pd.isna(x))]
['speed_limit'].unique())

Values that cannot be converted to int:
['<40']

```

<40 means less than 40, which is 'low' speed category, just set this value to 40.

```

df['speed_limit'] = df['speed_limit'].replace('<40', 40)

```

Categorising road **speed limits** into 4 categories. According to local rules in NT state speed limit in build-up areas is 60 km/h, while in other states is 50 km/h. We set "Low" for low speed zones 0-40 km/h, "Med" for 41-60 in NT and 41-50 in other states, 'High' for 61-80 in NT and 51-80 in other states, and 'Very High' for 81 and upper for all states.

```

df['speed_limit'] = np.where(
    df['state'] != "NT",
    np.select(
        [

```

```

        (df['speed_limit'] > 0) & (df['speed_limit'] <= 40),
        (df['speed_limit'] >= 41) & (df['speed_limit'] <= 50),
        (df['speed_limit'] >= 51) & (df['speed_limit'] <= 80),
        (df['speed_limit'] > 80)
    ],
    ['Low', 'Med', 'High', 'Very High'],
    default=np.nan
),
np.select(
    [
        (df['speed_limit'] > 0) & (df['speed_limit'] <= 40),
        (df['speed_limit'] >= 41) & (df['speed_limit'] <= 60),
        (df['speed_limit'] >= 61) & (df['speed_limit'] <= 80),
        (df['speed_limit'] > 80)
    ],
    ['Low', 'Med', 'High', 'Very High'],
    default=np.nan
)
)

# change 'nan' value to np.nan
df['speed_limit'] = df['speed_limit'].replace('nan', np.nan)

```

Create **vehicle_type** column, which indicates if there were heavy vehicles involved in the accident.

```

conditions = [
    (df['bus_involvement'] == "Yes") | (df['heavy_rigid_truck_involvement'] == "Yes") |
    (df['articulated_truck_involvement'] == "Yes"),
    (df['bus_involvement'] == "No") & (df['heavy_rigid_truck_involvement'] == "No") &
    (df['articulated_truck_involvement'] == "No")
]

choices = ["Heavy Vehicle Involved", "No Heavy Vehicle Involved"]

df['vehicle_type_involved'] = np.select(conditions, choices, default=np.nan)
df['vehicle_type_involved'] = df['vehicle_type_involved'].replace('nan', np.nan)

```

Transform population fact table from wide format to long format.

```
print(population.head())
```

Unnamed: 0	Unnamed: 1	2001	2002	2003	2004	2005	2006	\
0	LGA code Local Government Area	no.	no.	no.	no.	no.	no.	
1	10050	Albury	45265	45816	46180	46505	47004	47566
2	10180	Armidale	27906	27774	27610	27410	27350	27377
3	10250	Ballina	37856	38417	38870	39120	39305	39537
4	10300	Balranald	2751	2703	2661	2596	2545	2507

	2007	2008	...	2014	2015	2016	2017	2018	2019	2020	2021	\
0	no.	no.	...	no.	no.	no.	no.	no.	no.	no.	no.	
1	48140	48518	...	50990	51486	52171	53056	53922	54657	55466	56067	

```

2  27468  27788  ...  29015  29160  29310  29519  29631  29701  29600  29332
3  39824  40020  ...  41881  42336  42993  43652  44385  44997  45663  46196
4   2473   2433  ...   2376   2364   2330   2338   2308   2287   2257   2208

      2022   2023
0    no.    no.
1  56665  57517
2  29361  29594
3  46849  47279
4   2210   2202

[5 rows x 25 columns]

```

Create appropriate column names. Delete first row with 'no.' strings. Delete the last 2 rows with '© Commonwealth of Australia' and Total Australia measure.

```

# Clean the columnnames
year_colnames = population.columns[2:].tolist() # Get the first row for year column names
lga_colnames = population.iloc[0, :2].tolist() # Get the first two columns for LGA names

# change 'local_government_area' to 'lga_name'
lga_colnames[1] = 'lga_name'

for i in range(len(lga_colnames)):
    lga_colnames[i] = lga_colnames[i].strip().lower().replace(' ', '_').replace('/', '_')
population.columns = lga_colnames + year_colnames # Combine the two lists
population = population[1:-2].reset_index(drop=True)

```

Transform population into long format.

```

population_long = population.melt(
    id_vars = ["lga_code", "lga_name"],
    var_name = "year",
    value_name = "population"
)
population_long

lga_code  lga_name  year  population
0 10050 Albury   2001   45265
1 10180 Armidale 2001   27906
2 10250 Ballina 2001   37856
3 10300 Balranald 2001   2751
4 10470 Bathurst 2001  35504
... ..
12576 74660 West Arnhem 2023   7407
12577 74680 West Daly 2023   3426
12578 79399 Unincorporated NT 2023   7713
12579 89399 Unincorporated ACT 2023  466566
12580 99399 Unincorp. Other Territories 2023  2516
12581 rows x 4 columns

```

Designing dimension tables

To design the location dimension table, we needed to add `lga_code` to the data in the fatalities table. We merged the fatalities dataframe with the population table. After that, we discovered that some LGA names differed between the fatalities table and the population table (for example, *Armidale* vs. *Armidale Regional*). For these mismatches, we searched for matches based on the first word and updated the LGA names in the fact table so they would match the population table and avoid resulting in NaN values.

```
df = df.merge(population_long[['lga_name', 'lga_code']].drop_duplicates(), on='lga_name',
             how='left')

def match_by_first_word(row):
    # Only try to match if lga_code is missing and lga_name exists
    if pd.isna(row['lga_code']) and pd.notna(row['lga_name']):
        first_word = row['lga_name'].split()[0]
        # Try to find match in population_long
        match =
population_long[population_long['lga_name'].str.contains(rf'\b{first_word}\b', case=False,
na=False)]
        if not match.empty:
            # If a match is found, update both lga_code and lga_name
            row['lga_code'] = match['lga_code'].values[0]
            row['lga_name'] = match['lga_name'].values[0]
    return row

# Apply the function row-wise
df = df.apply(match_by_first_word, axis=1)
# make lga_code to int, ignore NaN
df['lga_code'] = df['lga_code'].astype(pd.Int64Dtype())
```

Create location dimension table.

```
location_dim = population_long[['lga_code', 'lga_name']].drop_duplicates()
location_dim = location_dim.merge(df[['lga_code', 'state']].drop_duplicates(),
on='lga_code', how='left')
location_dim = location_dim[['lga_code', 'state', 'lga_name']]
location_dim
```


	lga_code	state	lga_name
0	10050	NSW	Albury
1	10180	NSW	Armidale
2	10250	NSW	Ballina
3	10300	NSW	Balranald
4	10470	NSW	Bathurst
...	...	...	...
542	74660	NT	West Arnhem
543	74680	NT	West Daly
544	79399	NT	Unincorporated NT
545	89399	ACT	Unincorporated ACT
546	99399	NaN	Unincorp. Other Territories

547 rows x 3 columns

Create date dimension table.

```
date_dim = df[['year', 'month']].drop_duplicates()
date_dim['dateID'] = df['year'].astype(str) + df['month'].astype(str)
date_dim = date_dim[['dateID', 'year', 'month']]
date_dim
```

	dateID	year	month
0	202412	2024	12
113	202411	2024	11
244	202410	2024	10
359	20249	2024	9
454	20248	2024	8
...	...	...	...
55778	19895	1989	5
55991	19894	1989	4
56179	19893	1989	3
56435	19892	1989	2
56646	19891	1989	1

432 rows x 3 columns

Create Rush hours dimension table.

```

rush_dim = df[['time_cat']].drop_duplicates()
rush_dim['rushID'] = range(1, len(rush_dim) + 1)
rush_dim = rush_dim[['rushID', 'time_cat']]
rush_dim.dropna(subset=['time_cat'], inplace=True)
rush_dim

```

```

   rushID  time_cat
0      1    Not Rush
7      2      Rush

```

	rushID	time_cat
0	1	Not Rush
7	2	Rush

Create Age dimension table.

```

age_dim = df[['age_group']].drop_duplicates()
# sort by age_group
age_dim = age_dim.sort_values(by='age_group')
age_dim['ageID'] = range(1, len(age_dim) + 1)
age_dim = age_dim[['ageID', 'age_group']]
age_dim.dropna(subset=['age_group'], inplace=True)
age_dim

```

	ageID	age_group
20	1	0_to_16
1	2	17_to_25
2	3	26_to_39
4	4	40_to_64
0	5	65_to_74
18	6	75_or_older

Create Daytime dimension table.

```

daytime_dim = df[['time_of_day']].drop_duplicates()
daytime_dim['daytimeID'] = range(1, len(daytime_dim) + 1)
daytime_dim = daytime_dim[['daytimeID', 'time_of_day']]
daytime_dim.dropna(subset=['time_of_day'], inplace=True)
daytime_dim

```

	daytimeID	time_of_day
0	1	Night
1	2	Day

Create Road dimension table.

```

road_type_dim = df[['road_type']].drop_duplicates()
road_type_dim['road_typeID'] = range(1, len(road_type_dim) + 1)
road_type_dim = road_type_dim[['road_typeID', 'road_type']]
road_type_dim.dropna(subset=['road_type'], inplace=True)
road_type_dim

```

road_typeID		road_type
0	1	Arterial Road
1	2	Local Road
3	3	National or State Highway
6	5	Sub-arterial Road
17	6	Collector Road
154	7	Pedestrian Thoroughfare
283	8	Access road
970	9	Busway

Create Speed dimension table.

```

speed_limit_dim = df[['speed_limit']].drop_duplicates().dropna()
speed_limit_dim['speed_limitID'] = range(1, len(speed_limit_dim) + 1)
speed_limit_dim = speed_limit_dim[['speed_limitID', 'speed_limit']]
speed_limit_dim.dropna(subset=['speed_limit'], inplace=True)
speed_limit_dim

```

speed_limitID		speed_limit
0	1	Very High
1	2	High
2	3	Med
25	4	Low

Create Holiday dimension table.

```

holiday_dim = df[['holiday_type',
'holiday_name']].drop_duplicates().sort_values(by='holiday_type')
holiday_dim['holidayID'] = range(1, len(holiday_dim) + 1)
holiday_dim = holiday_dim[['holidayID', 'holiday_type', 'holiday_name']]
holiday_dim.dropna(subset=['holiday_type', 'holiday_name'], inplace=True)
holiday_dim

```

holidayID		holiday_type	holiday_name
0	1	Holiday	Christmas
892	2	Holiday	Easter
1	3	NoHoliday	NoHoliday

Create Vehicle type dimension table.

```

vehicle_type_dim = df[['vehicle_type_involved']].drop_duplicates().dropna()
vehicle_type_dim['vehicle_typeID'] = range(1, len(vehicle_type_dim) + 1)
vehicle_type_dim = vehicle_type_dim[['vehicle_typeID', 'vehicle_type_involved']]
vehicle_type_dim.dropna(subset=['vehicle_type_involved'], inplace=True)
vehicle_type_dim

```

vehicle_typeID		vehicle_type_involved
0	1	No Heavy Vehicle Involved
13	2	Heavy Vehicle Involved

Load

Saving tables

Save all dimension tables to csv files.

```

dimension_tables = {
    "location_dim": location_dim,
    "date_dim": date_dim,
    "rush_dim": rush_dim,
    "age_dim": age_dim,
    "daytime_dim": daytime_dim,
    "road_type_dim": road_type_dim,
    "speed_limit_dim": speed_limit_dim,
    "holiday_dim": holiday_dim,
    "vehicle_type_dim": vehicle_type_dim
}
# Save each DataFrame to a CSV file
for name, table in dimension_tables.items():
    output_file_path = f"data/{name}.csv"
    table.to_csv(output_file_path, index=False)
    print(f"{name} data saved to {output_file_path}")

```

Save the cleaned DataFrame for Algorithm Rule Mining.

```
df_cleaned = df.copy()
```

Create fatalities_df fact table.

```

fatalities_df = df.copy()
fatalities_df = fatalities_df.merge(date_dim, on=['year', 'month'], how='left')
fatalities_df = fatalities_df.merge(rush_dim, on=['time_cat'], how='left')
fatalities_df = fatalities_df.merge(age_dim, on=['age_group'], how='left')
fatalities_df = fatalities_df.merge(daytime_dim, on=['time_of_day'], how='left')
fatalities_df = fatalities_df.merge(road_type_dim, on=['road_type'], how='left')
fatalities_df = fatalities_df.merge(speed_limit_dim, on=['speed_limit'], how='left')
fatalities_df = fatalities_df.merge(holiday_dim, on=['holiday_type', 'holiday_name'],
how='left')

```

```
fatalities_df = fatalities_df.merge(vehicle_type_dim, on=['vehicle_type_involved'],
how='left')

# drop unnecessary columns
fatalities_df = fatalities_df[['crash_id', 'victim_number', 'lga_code', 'dateID',
'rushID', 'ageID', 'daytimeID', 'road_typeID', 'speed_limitID', 'holidayID',
'vehicle_typeID']]
# Convert all values to Int64 ingnoring NaN
fatalities_df = fatalities_df.astype(pd.Int64Dtype())
fatalities_df
```

	crash_id	victim_number	lga_code	dateID	rushID	ageID	daytimeID	road_typeID	speed_limitID	holidayID	vehicle_typeID
0	20241115	1	17750	202412	1	5	1	1	1	1	1
1	20241125	1	13800	202412	1	2	2	2	2	3	1
2	20246013	1	64610	202412	1	3	2	2	3	1	1
3	20241002	1	10180	202412	1	3	2	3	1	3	1
4	20242261	1	<NA>	202412	1	4	2	<NA>	<NA>	3	<NA>
...	...	...	...	...	...	...	...	...	...	...	...
56869	19896006	3	<NA>	19891	1	1	1	<NA>	1	3	2
56870	19896006	4	<NA>	19891	1	1	1	<NA>	1	3	2
56871	19896006	5	<NA>	19891	1	2	1	<NA>	1	3	2
56872	19896006	6	<NA>	19891	1	1	1	<NA>	1	3	2
56873	19895133	1	<NA>	19891	1	5	1	<NA>	<NA>	3	<NA>

56874 rows x 11 columns

Create population_df fact table.

```
population_df = population_long.copy()
population_df = population_df[['lga_code', 'year', 'population']]
population_df
```

	lga_code	year	population
0	10050	2001	45265
1	10180	2001	27906
2	10250	2001	37856
3	10300	2001	2751
4	10470	2001	35504
...	...	...	...
12576	74660	2023	7407
12577	74680	2023	3426
12578	79399	2023	7713
12579	89399	2023	466566
12580	99399	2023	2516

12581 rows x 3 columns

Save fact tables.

```

fact_tables = {
    "fatalities_fact": fatalities_df,
    "population_fact": population_df
}
# Save each DataFrame to a CSV file
for name, table in fact_tables.items():
    output_file_path = f"data/{name}.csv"
    table.to_csv(output_file_path, index=False)
    print(f"{name} data saved to {output_file_path}")

```

Load tables to PostgreSQL

In the Data Warehouse we create new database Project_1 and create tables with SQL queries:

```

-- DROP ALL TABLES
DROP TABLE IF EXISTS population_fact;
DROP TABLE IF EXISTS fatalities_fact;
DROP TABLE IF EXISTS location_dim;
DROP TABLE IF EXISTS date_dim;
DROP TABLE IF EXISTS rush_dim;
DROP TABLE IF EXISTS age_dim;
DROP TABLE IF EXISTS daytime_dim;
DROP TABLE IF EXISTS road_type_dim;
DROP TABLE IF EXISTS speed_limit_dim;
DROP TABLE IF EXISTS holiday_dim;
DROP TABLE IF EXISTS vehicle_type_dim;
DROP TABLE IF EXISTS population_fact;
DROP TABLE IF EXISTS fatalities_fact;

-- create dimension location_dim
CREATE TABLE IF NOT EXISTS location_dim (
    lga_code INT PRIMARY KEY,
    state VARCHAR(50),
    lga_name VARCHAR(100)
);

-- create dimension date_dim
CREATE TABLE IF NOT EXISTS date_dim (
    dateID INT PRIMARY KEY,
    year INT,
    month INT
);

-- create dimension rush_dim
CREATE TABLE IF NOT EXISTS rush_dim (
    rushID INT PRIMARY KEY,
    time_cat VARCHAR(20)
);

-- create dimension age_dim
CREATE TABLE IF NOT EXISTS age_dim (

```

```

    ageID INT PRIMARY KEY,
    age_group VARCHAR(20)
);

-- create dimension daytime_dim
CREATE TABLE IF NOT EXISTS daytime_dim (
    daytimeID INT PRIMARY KEY,
    time_of_day VARCHAR(20)
);

-- create dimension road_type_dim
CREATE TABLE IF NOT EXISTS road_type_dim (
    road_typeID INT PRIMARY KEY,
    road_type VARCHAR(30)
);

-- create dimension speed_limit_dim
CREATE TABLE IF NOT EXISTS speed_limit_dim (
    speed_limitID INT PRIMARY KEY,
    speed_limit VARCHAR(20)
);

-- create dimension holiday_dim
CREATE TABLE IF NOT EXISTS holiday_dim (
    holidayID INT PRIMARY KEY,
    holiday_type VARCHAR(50),
    holiday_name VARCHAR(50)
);

-- create dimension vehicle_type_dim
CREATE TABLE IF NOT EXISTS vehicle_type_dim (
    vehicle_typeID INT PRIMARY KEY,
    vehicle_type_involved VARCHAR(50)
);

-- create population fact table
CREATE TABLE IF NOT EXISTS population_fact (
    lga_code INT,
    year INT,
    population INT,
    FOREIGN KEY (lga_code) REFERENCES location_dim(lga_code),
    PRIMARY KEY (lga_code, year)
);

-- create fatalities fact table
CREATE TABLE IF NOT EXISTS fatalities_fact(
    crash_ID INT,
    victim_number INT,
    lga_code INT,
    dateID INT,
    rushID INT,
    ageID INT,

```

```

daytimeID INT,
road_typeID INT,
speed_limitID INT,
holidayID INT,
vehicle_typeID INT,
FOREIGN KEY (lga_code) REFERENCES location_dim(lga_code),
FOREIGN KEY (dateID) REFERENCES date_dim(dateID),
FOREIGN KEY (rushID) REFERENCES rush_dim(rushID),
FOREIGN KEY (ageID) REFERENCES age_dim(ageID),
FOREIGN KEY (daytimeID) REFERENCES daytime_dim(daytimeID),
FOREIGN KEY (road_typeID) REFERENCES road_type_dim(road_typeID),
FOREIGN KEY (speed_limitID) REFERENCES speed_limit_dim(speed_limitID),
FOREIGN KEY (holidayID) REFERENCES holiday_dim(holidayID),
FOREIGN KEY (vehicle_typeID) REFERENCES vehicle_type_dim(vehicle_typeID),
PRIMARY KEY (crash_ID, victim_number)

```

Load data into the tables.

```

DELETE FROM fatalities_fact;
DELETE FROM population_fact;
DELETE FROM location_dim;
DELETE FROM date_dim;
DELETE FROM rush_dim;
DELETE FROM age_dim;
DELETE FROM daytime_dim;
DELETE FROM road_type_dim;
DELETE FROM speed_limit_dim;
DELETE FROM holiday_dim;
DELETE FROM vehicle_type_dim;

COPY location_dim FROM '/tmp/location_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY date_dim FROM '/tmp/date_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY rush_dim FROM '/tmp/rush_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY age_dim FROM '/tmp/age_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY daytime_dim FROM '/tmp/daytime_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY road_type_dim FROM '/tmp/road_type_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY speed_limit_dim FROM '/tmp/speed_limit_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY holiday_dim FROM '/tmp/holiday_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY vehicle_type_dim FROM '/tmp/vehicle_type_dim.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY fatalities_fact FROM '/tmp/fatalities_fact.csv' WITH (FORMAT CSV, HEADER TRUE);
COPY population_fact FROM '/tmp/population_fact.csv' WITH (FORMAT CSV, HEADER TRUE);

```

Visualisation

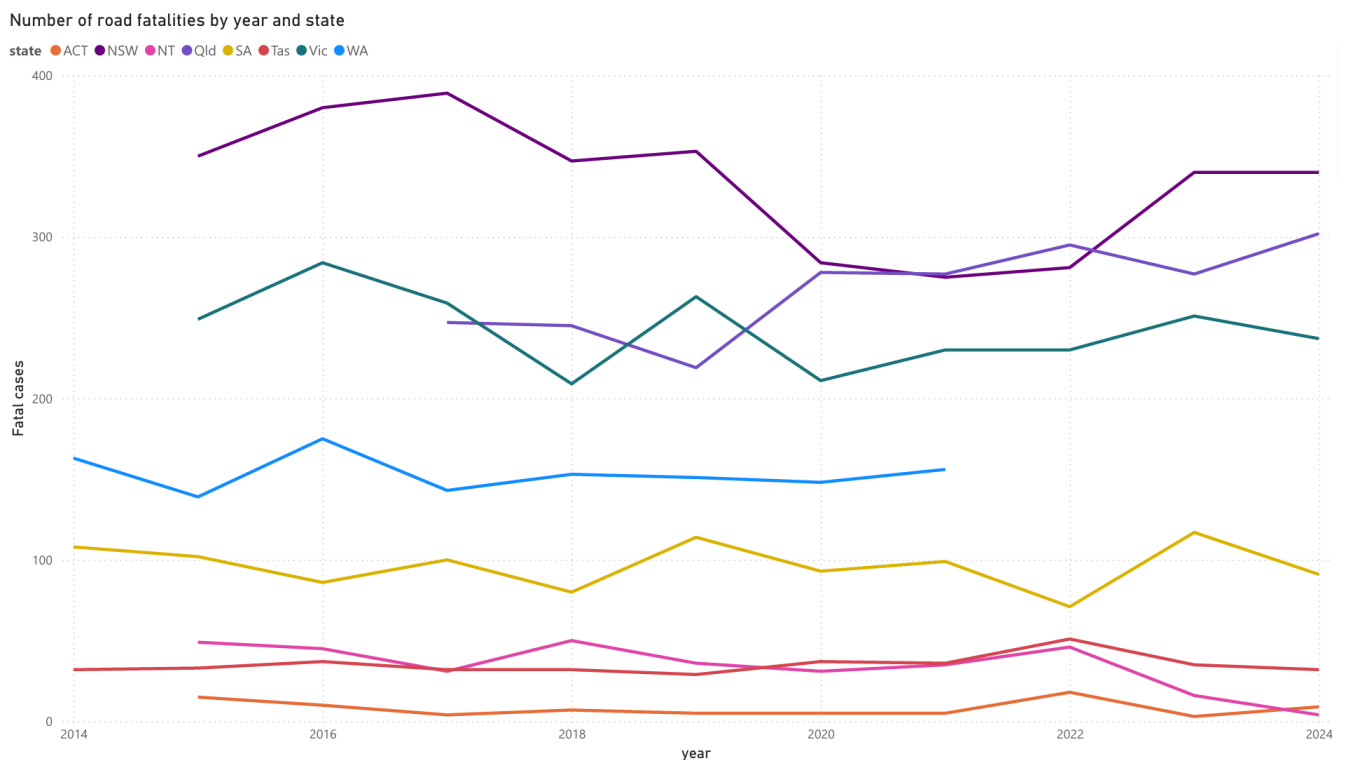
1. What is the total number of road fatalities by state and year?


```

SELECT
    d.year,
    l.state,
    COUNT(*) AS total_fatalities
FROM fatalities_fact f
JOIN date_dim d ON f.dateID = d.dateID
JOIN location_dim l ON f.lga_code = l.lga_code

GROUP BY ROLLUP (d.year, l.state);

```



As you can see on this line graph, the distribution of fatal road accident incidents by state by year is presented here. The first lines in the number of deaths at the beginning are occupied by NSW, Vic, WA, which is logical since these are some of the most populated states in Australia, however, the state of Qld, which will take 3rd place in the future, has not yet kept statistics on road accidents. At the end of the dataset, the lines are logically distributed between the three largest states in Australia NSW, Qld and Vic. In the period between 2019 and 2021, when the pandemic occurred, an interesting situation is observed when in the leading state in terms of the number of deaths NSW this figure fell by almost 25%, while in Qld this figure, on the contrary, increased by 20%.

2. How many road crashes involving heavy vehicles occurred during the day and night across each state?

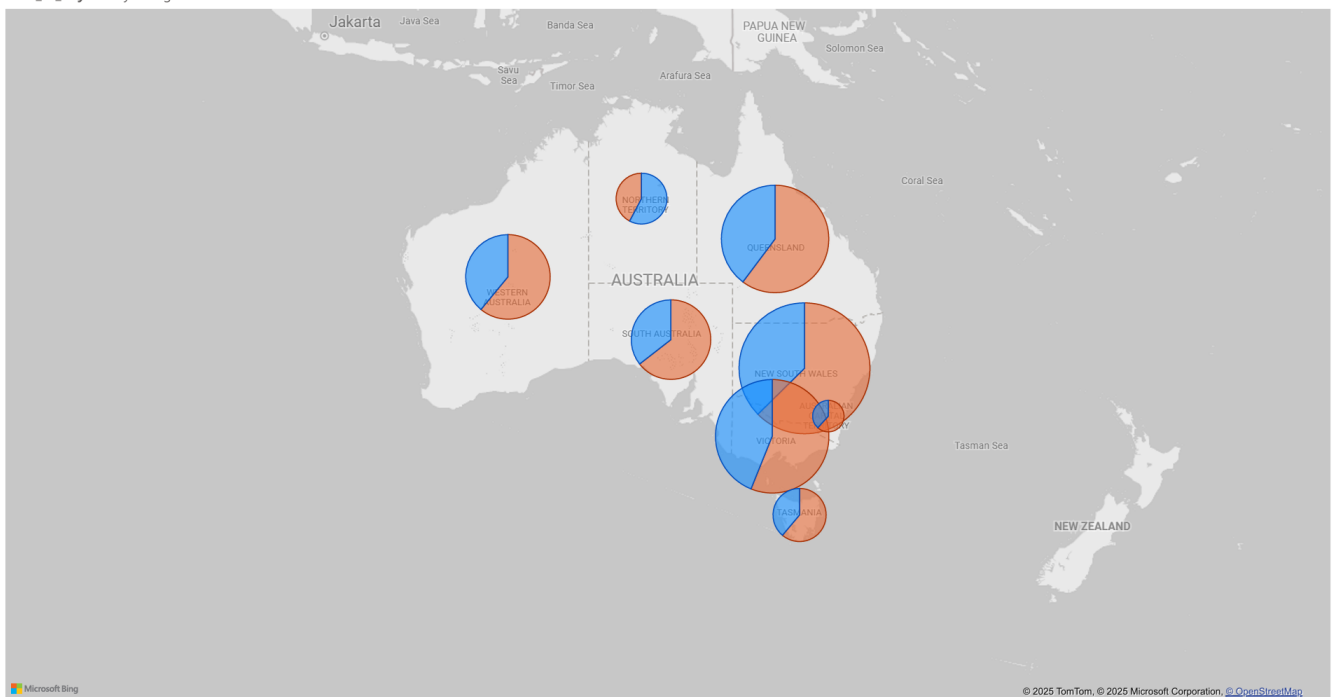
```

SELECT
    l.state,
    d.time_of_day,
    COUNT(*) AS total_crashes
FROM fatalities_fact f
JOIN location_dim l ON f.lga_code = l.lga_code
JOIN daytime_dim d ON f.daytimeID = d.daytimeID
JOIN vehicle_type_dim v ON f.vehicle_typeID = v.vehicle_typeID
WHERE v.vehicle_type_involved = 'Heavy Vehicle Involved'
GROUP BY l.state, d.time_of_day
ORDER BY l.state, d.time_of_day;

```

How many road crashes by State and Daytime

time_of_day ● Day ● Night



This geographic map of Australia overlays pie charts showing the distribution of road accidents during the day and at night in different states. In general, the ratio between day and night time is the same in most states. However, in the state of Vic, there is a slight increase in the share of night accidents, while in the state of NT, there is an anomalous situation when, on the contrary, there are much more night accidents than daytime ones, which is not observed anywhere else, which may indicate a special situation or large structural differences from other states.

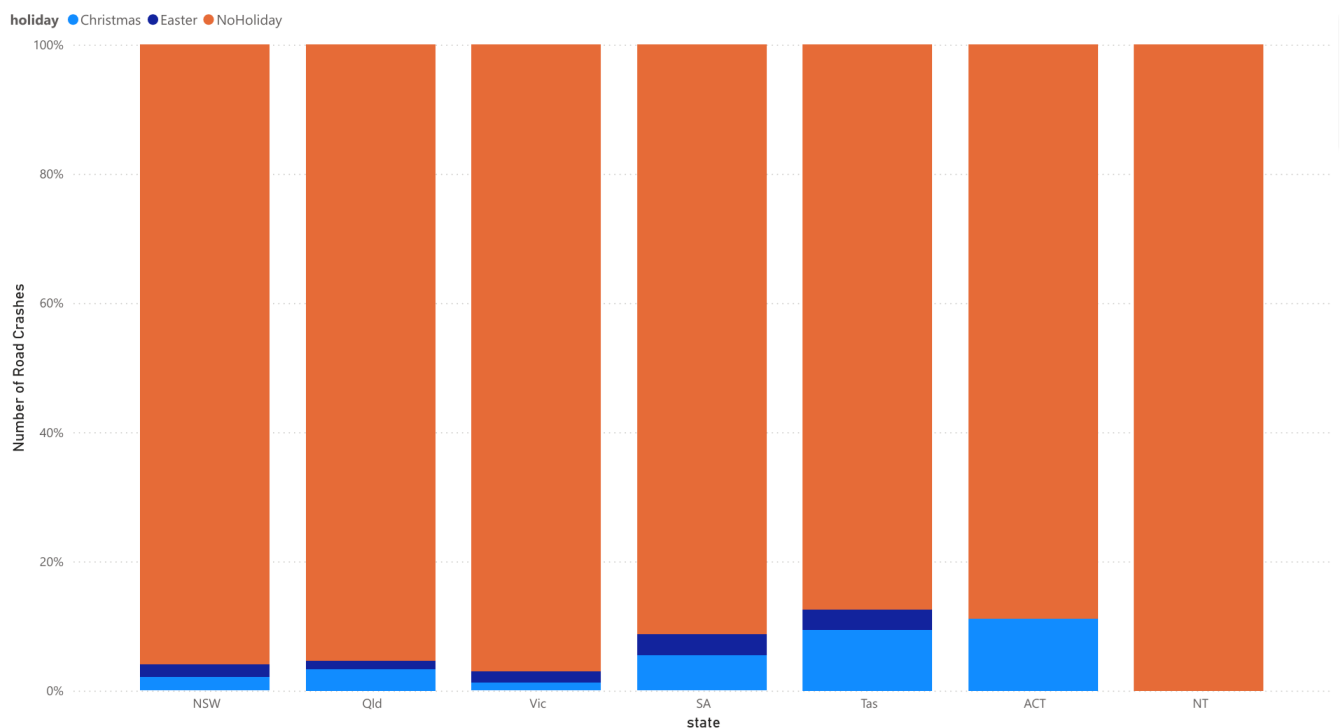
3. How many road crashes occurred during holidays in each state in 2024?

```

SELECT
    l.state,
    h.holiday_type,
    COUNT(*) AS total_crashes
FROM fatalities_fact f
JOIN location_dim l ON f.lga_code = l.lga_code
JOIN holiday_dim h ON f.holidayID = h.holidayID
JOIN date_dim d ON f.dateID = d.dateID
WHERE d.year = 2024
AND h.holiday_type = 'Holiday'
GROUP BY l.state, h.holiday_type
ORDER BY total_crashes DESC;

```

How many road crashes occurred during holidays in each state in 2024



This graph shows the relative shares of accidents on weekdays and holidays (Christmas, Easter) in different states in 2024. Some states have good normal shares when there are much fewer people killed on holidays than on weekdays. In other states NT, WA, SA, TA, ACT, due to the small number of accidents, there are distorted statistics where there were never any accidents before the strong increase in the share of holidays.

4. Which age group is most frequently involved in road crashes during rush hours?

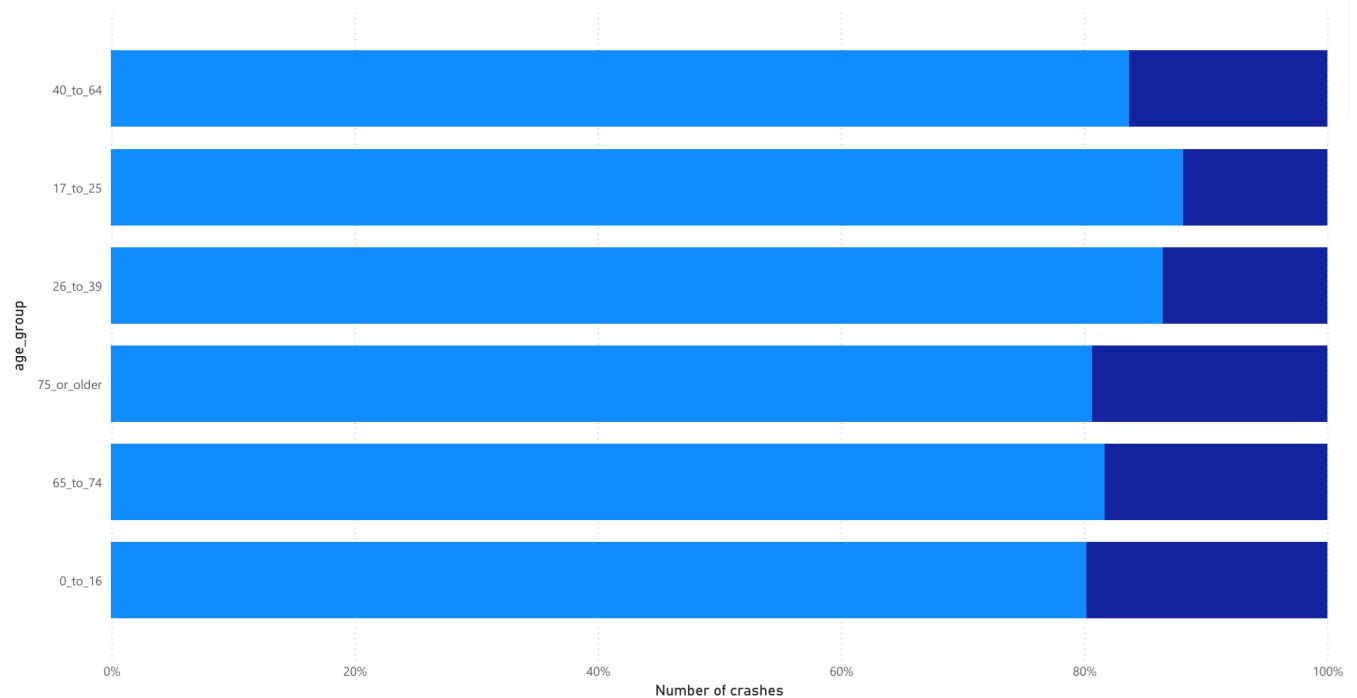
```

SELECT
    a.age_group,
    COUNT(*) AS total_crashes
FROM fatalities_fact f
JOIN rush_dim r ON f.rushID = r.rushID
JOIN age_dim a ON f.ageID = a.ageID
WHERE r.time_cat = 'Rush'
GROUP BY a.age_group
ORDER BY total_crashes DESC;

```

Which age group is most frequently involved in road crashes during rush hours

time_cat ● Not Rush ● Rush



This chart shows different age groups and how often they get into accidents during rush hour.

Overall, people aged 0 to 16 and 65 and older get into accidents during rush hour on average 20% of the time. While people aged 17 to 64 have a chance of getting into an accident during rush hour of about 14%.

5. How many road crashes occurred at each speed limit in each Australian state during the year 2024?

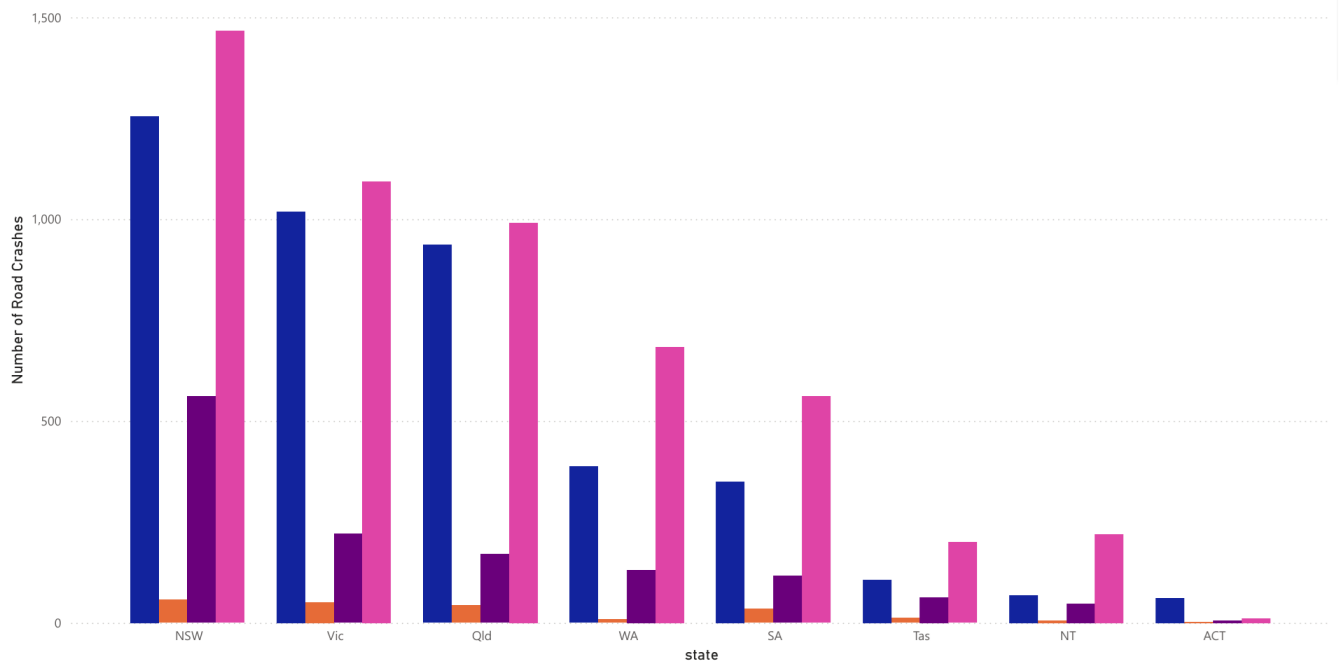
```

SELECT
    l.state,
    s.speed_limit,
    COUNT(*) AS total_crashes
FROM fatalities_fact f
JOIN location_dim l ON f.lga_code = l.lga_code
JOIN speed_limit_dim s ON f.speed_limitID = s.speed_limitID
JOIN date_dim d ON f.dateID = d.dateID
WHERE d.year = 2024
GROUP BY l.state, s.speed_limit
ORDER BY l.state, total_crashes DESC;

```

How many road crashes occurred at each speed limit in each Australian state during the year 2024

speed_limit ● High ● Low ● Med ● Very High



This bar chart shows the number of accidents in different states in 2024 at different speeds. As expected, most fatal accidents occur at high and very high speeds. At the same time, the number of accidents at low and medium speeds (up to 50-60 km / h) make up only a small share of all accidents.

6. What is the road fatality rate per 100,000 people by state in the year 2020?

```

WITH population_per_state AS (
    SELECT
        l.state,
        SUM(p.population) AS total_population
    FROM population_fact p
    JOIN location_dim l ON p.lga_code = l.lga_code
    WHERE p.year = 2020
    GROUP BY l.state
)

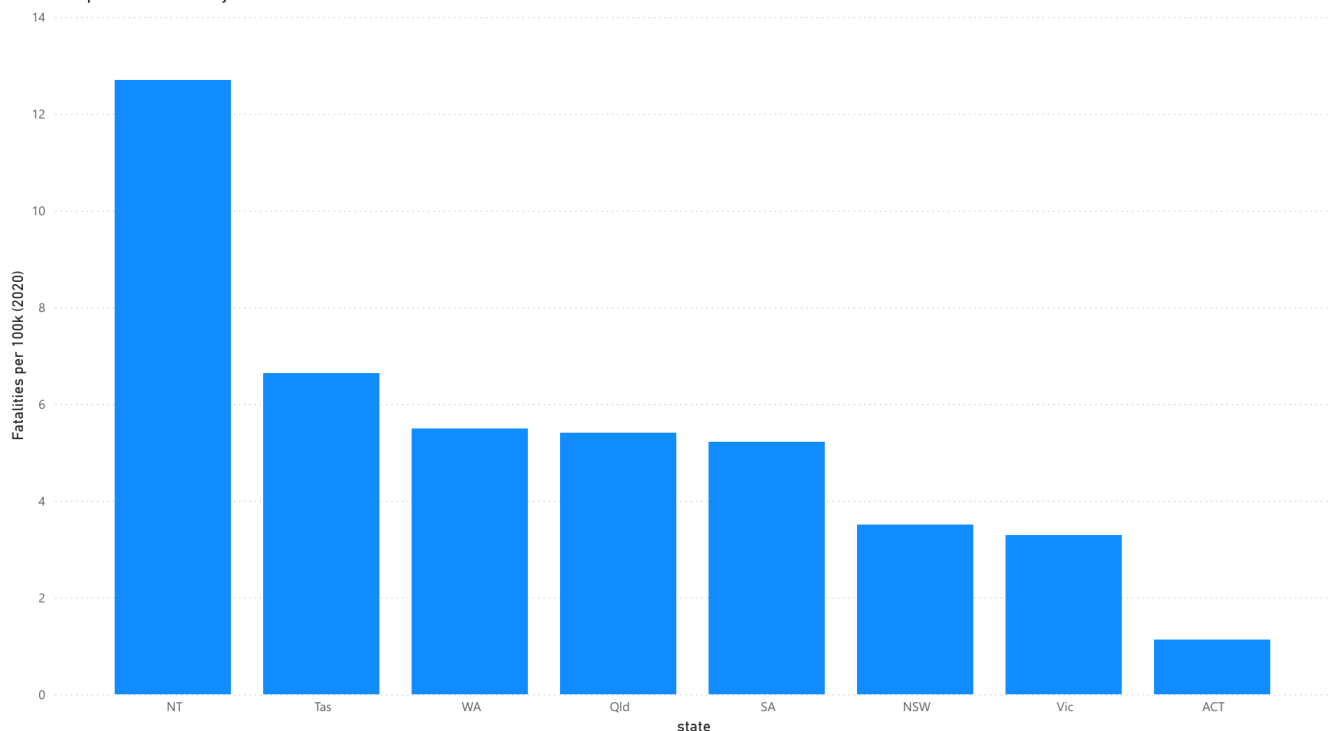
```

```

),
fatalities_per_state AS (
    SELECT
        l.state,
        COUNT(f.crash_ID) AS total_deaths
    FROM fatalities_fact f
    JOIN location_dim l ON f.lga_code = l.lga_code
    JOIN date_dim d ON f.dateID = d.dateID
    WHERE d.year = 2020
    GROUP BY l.state
)
SELECT
    pps.state,
    pps.total_population,
    fps.total_deaths,
    ROUND(fps.total_deaths * 100000.0 / NULLIF(pps.total_population, 0), 2) AS
fatalities_per_100k
FROM population_per_state pps
JOIN fatalities_per_state fps ON pps.state = fps.state
ORDER BY fatalities_per_100k DESC;

```

Fatalities per 100k (2020) by state



This graph shows the mortality rate from road accidents in different states in 2020, calculated per 100 thousand people. The best indicators are demonstrated by the state of ACT. NSW and Vic have very similar metrics and in general they are better than the national average. At the same time, an abnormal situation is again observed in the state of NT. The road mortality rate in this state is almost 3 times longer than in NSW and Vic and almost twice as much as the state average. This metric may indicate major problems with either road infrastructure or road administrative regulation.

Association rules mining

1.Explanation and discussion which association rules mining algorithms were used.

Algorithm Name	Pros	Cons
Apriori	quite easy to understand the process. Works faster with a large number of columns and high sparseness of data	With a large number of rows, it creates an excessive number of candidates. It works slower than alternatives with a small number of features.
Fp-Growth	Creates fewer candidates due to the tree construction process itself with a large number of rows. Overall, shows better performance for most datasets	Can slow down significantly with a large number of columns and very sparse data

Apriori:

The main concept in this algorithm that's it iteratively grows frequent itemsets by going through the data many times. First, it looks for frequent 1-itemsets, then 2-items based on the 1-items

Fp-Growth:

It builds a prefix tree (FP-tree) for a compressed representation of the data. Then recursively "unwinds" the tree, extracting frequent sets without multiple scans of the entire dataframe.

References:

Ali, M. (2023, January). *Association Rule Mining in Python Tutorial*. datacamp. <https://www.datacamp.com/tutorial/association-rule-mining-python>

R. Agrawal, R. Srikant. "Fast algorithms for mining association rules." *Proceedings of the 20th VLDB Conference*, 1994.

Li, H., Wang, Y., Zhang, D., Zhang, M., & Chang, E. Y. (2008, October). Pfp: parallel fp-growth for query recommendation. In *Proceedings of the 2008 ACM conference on Recommender systems*, 107-114.

First, we need to prepare a binary matrix for the algorithms. The column is divided into columns of dummy variables so that each column has its own binary column for each of the feature values. However, we decided not to take all the columns for this, since they seemed insignificant to us compared to how much they load the calculations. According to this principle, we did not take into account the features year, month, lga_name. The final dataset included the columns:

```
state', 'speed_limit', 'holiday', 'time_of_day', 'time_cat', 'road_user', 'vehicle_type_involved', 'road_type', 'age_group'
```

Our final matrix included:

```
[56874 rows x 48 columns]
```

In our data study, we decided to initially try two algorithms and decide based on the results which one suits us best. Both algorithms are implemented through the `mlxtend.frequent_patterns` module.

Before we start implementing the algorithm, we need to decide what level of minimum support we need to use. To do this, we use the simplest filter for all values of the `road_user` feature to find out the distribution of different categories within the group.

```
encoded_df.filter(like='road_user_').mean()
```

Our output is:

road_user_Driver	0.451542
road_user_Motorcycle pillion passenger	0.006752
road_user_Motorcycle rider	0.131097
road_user_Passenger	0.227134
road_user_Pedal cyclist	0.027183
road_user_Pedestrian	0.153972
road_user_nan	0.002321

As we can see, the most frequent category is Driver, which occurs in almost 45% of all cases. At the same time, other options are much less common. We decided to use 10% as a starting value because if we took more, for example 15~20%, then we would most likely not see any associations for the Motorcycle rider and Pedestrian options, since they would occur in the dataset less often than our minimum support percentage. However, after a series of tests, we settled on a parameter equal to 5%, since with it we do not have a small set of obvious patterns as with 10~15%, but at the same time we do not have a lot of white noise and random patterns as with 0.005~0.03%. With this parameter, we get the golden mean when we do not refuse all rare interesting finds, but at the same time we do not get a large amount of white noise.

```
min_support = 0.05
```

For confidence, we decided to start with the standard 50%. Since we did not want to set strict rules right away, and especially if something is observed on a large dataset with a probability of more than 50~60% with a sufficiently high lift, then this already deserves attention, because this finding is already very likely not a simple coincidence and requires attention. Also, for convenience, we rank all the found rules at the end by lift and confidence in descending order, which will still give us the best finds right from the top, but at the same time leave us with the opportunity to see rarer finds with probabilities of 50~70%.

```
min_confidence = 0.5
```

Below are examples of how we implemented the Apriori and FP-Growth algorithms in practice.

```
#Apriori
min_support = 0.05
```



```

frequent_itemsets_ap = apriori(encoded_df, min_support=min_support, use_colnames=True,
low_memory=True)
#low_memory we used because Its allow process data by chucnks avoiding any Memory
overusage crashes

print("Itemsets")
print(frequent_itemsets_ap.head())

min_confidence = 0.5
rules = association_rules(frequent_itemsets_ap, metric="confidence",
min_threshold=min_confidence)

filtered_rules_ap = rules[
    (rules['consequents'].apply(lambda x: len(x) == 1 and
next(iter(x)).startswith('road_user_'))))
]

rules = rules.sort_values(by=['lift', 'confidence'], ascending=[False, False])#Just to see
other rules for
    # other categories
filtered_rules_ap = filtered_rules_ap.sort_values(by=['lift', 'confidence'], ascending=
[False, False])

print("\n(Apriori):")
print(rules.head(5))

print("\n K-5 rules")
print(filtered_rules_ap.head(5))

```

```

#Fp-Growth
min_support = 0.05

frequent_itemsets_fp = fpgrowth(encoded_df, min_support=min_support, use_colnames=True)

print("Itemsets")
print(frequent_itemsets_fp.head())

min_confidence = 0.5
rules_fp = association_rules(frequent_itemsets_fp, metric="confidence",
min_threshold=min_confidence)

filtered_rules_fp = rules_fp[
    (rules_fp['consequents'].apply(lambda x: len(x) == 1 and
next(iter(x)).startswith('road_user_'))))
]

rules_fp = rules_fp.sort_values(by=['lift', 'confidence'], ascending=[False, False])
filtered_rules_fp = filtered_rules_fp.sort_values(by=['lift', 'confidence'], ascending=
[False, False])

```

```
print("\n(Fp-Growth):")
print(rules_fp.head(5))

print("\n K-5 rules:")
print(filtered_rules_fp.head(5))
```

In the process of testing different combinations of parameters and features, we came to a situation in which we had to abandon the Fp-Growth algorithm, since in our conditions it worked much slower than Apriori, despite the fact that the top rules were almost always the same. Due to the greater performance of Apriori, we ultimately based all our answers on it.

2-3. The top k rules which we received and explanation the rules meaning

antecedents	consequents	confidence	lift
speed_limit_Very High, vehicle_type_involved_Heavy Vehicle Involved	Driver	0.645357986	1.429231342
speed_limit_Very High, holiday_type_NoHoliday, vehicle_type_involved_Heavy Vehicle Involved	Driver	0.644448906	1.427218063
speed_limit_Very High, age_group_40_to_64, holiday_type_NoHoliday, 'road_type_nan	Driver	0.621307667	1.375968702
speed_limit_Very High, holiday_type_NoHoliday, age_group_26_to_39	Driver	0.620422098	1.374007492
speed_limit_Very High, road_type_nan, age_group_40_to_64	Driver	0.619792498	1.372613159

Rule #1:

Antecedents: (speed_limit_Very High, vehicle_type_involved_Heavy Vehicle Involved)

Consequents: (Driver)

Confidence: 64.5%, Lift: 1.43

Interpretation:

Of all records (accidents), approximately 64.5% of the time when the combination of "very high speed limit" and "heavy vehicle involved" is recorded, the victim is a driver. Since lift = 1.43 (> 1), the presence of "very high speed" and "heavy vehicle" increases the probability of driver involvement by 43% compared to random distribution.

Rule #2:

Antecedents: (speed_limit_Very High, holiday_type_NoHoliday, vehicle_type_involved_Heavy Vehicle Involved)

Consequents: (Driver)

Confidence: 64.4%, Lift: 1.43

Interpretation:

Of all accidents that have the combination of "very high speed limit", "normal (non-holiday) day" and "heavy vehicle involved", approximately 64.4% of the time a driver is involved. Since lift = 1.43 (> 1), this means that this combination of conditions increases the probability that the driver will be involved in an accident by 43% compared to typical conditions.

Rule #3:

Antecedents: (speed_limit_Very High, age_group_40_to_64, holiday_type_NoHoliday, road_type_nan)

Consequents: (Driver)

Confidence: 62.1%, Lift: 1.38

Interpretation:

Of all crashes, approximately 62.1% of the cases where the conditions "very high speed limit", "driver age group 40 to 64", "not a holiday", and "road type not specified (omitted)" are all met involve a driver participant. Since lift = 1.38 (> 1), the presence of this combination of conditions increases the probability of driver involvement by 38% above chance.

Rule #4:

Antecedents: (speed_limit_Very High, holiday_type_NoHoliday, age_group_26_to_39)

Consequents: (Driver)

Confidence: 62.0%, Lift: 1.37

Interpretation:

When the data contains the conditions "very high speed limit", "not a holiday" and "driver age from 26 to 39 years old", in 62.0% of cases the accidents involve drivers. Lift = 1.37 (> 1), that is, this combination of conditions increases the probability that the driver will be involved in the accident by 37%, compared to the usual situation.

Rule #5:

Antecedents: (speed_limit_Very High, road_type_nan, age_group_40_to_64)

Consequents: (Driver)

Confidence: 61.9%, Lift: 1.37

4. Insights and explanation the reasons for our outcome.

In crashes where the combination of "very high speed limit", "road type not specified (omitted)" and "age group from 40 to 64 years" is present, approximately 61.9% of the time the driver is involved. Lift = 1.37 (> 1) shows that this combination increases the probability of driver involvement by 37% relative to the overall probability.

Very High speed limit is one of the most significant factors associated with fatal accidents of drivers. Almost all rules with high lift and confidence include this feature. This suggests that high speeds are one of the main reasons for most fatal accidents for drivers.

It is a logical observation that the age group (26-64) is often encountered. However, this is logical because in the population section this age group is the most common in Australia. This age is especially often involved in fatal accidents in combination with heavy transport.

Holidays are not a risk factor in these rules since the most significant rules concern non-holidays. As a result, holiday trips do not entail an increased likelihood of getting into an accident for the main group of drivers.

The participation of heavy transport is an important feature. If it is present, the probability of the driver's participation in an accident is significantly higher. This may reflect accidents in the city with trucks and buses, as well as accidents with logistics transport on intercity routes.

The missing road type (road_type_nan) is often found in the rules. This is caused by incomplete data and frequent omissions in this column over the years. This could be eliminated by filtering all omissions or even completely deleting rows with omissions, but then we would have a chance to encounter data distortion and incorrect representation.

5.Recommendations to the government.

1. The first recommendation is to introduce stricter control and penalties for speeding. It is also important to design and modify the existing structure to modern standards, taking into account the practices of implicit speed control and the installation of protective barriers on roads where drivers can potentially exceed the speed limit. Particular attention to speeding drivers should be introduced in the states of NSW and VIC
2. Improve the overall safety of roads inside the city by creating separate lanes for buses, redirecting the flow of trucks to bypass the city if possible. And also widening intercity roads to reduce the number of accidents when car drivers try to overtake slow trucks by driving into the oncoming lane on narrow roads.
3. Although the top 5 rules did not include the reason for night trips, further down the list you can find a large number of important rules indicating the problem of night roads as one of the key factors of mortality for drivers. To correct this situation, it is necessary to improve night markings and road lighting. It is even possible to introduce control over the brightness of headlights on cars during regular technical inspections in order to prevent situations when too bright headlights blind all drivers in front of them and vice versa when too dim headlights do not provide sufficient visibility. Particular attention should be paid to this problem in the state of NT.